

SIEGE SOCIAL :
ZIRST 12 CHEMIN DE MALACHER
B.P. 85
38243 MEYLAN CEDEX
FRANCE

TEL. : +33 (0)4 76 90 72 72
TELEFAX : +33 (0)4 76 41 00 83
E-mail : postmaster@gea.fr URL : <http://www.gea.fr>
S.A. AU CAPITAL DE 2.400.000 Euros
R.C.S. GRENOBLE B 071 501 803 00028
APE 333 Z - N°TVA C.E.E. FR 09 071 501 803



GEA

BEACON MANAGER

USER MANUAL

ISSUE : SEPTEMBER 2010

UPDATES

Index	Scope	Classification
A	Creation.	Project
B	Add the management of TGR-IP	

Index	Written by Date / Visa	Checked by Date / Visa	Approved by Date / Visa
A	C.SALENC 16/08/2010	C.SALENC	C.SALENC
B	C.REYMOND 24/09/2010	C.SALENC	C.SALENC

CONTENTS

1. INTRODUCTION	4
2. API REFERENCE	5
3. DLL MANAGEMENT	6
3.1. READ THE DLL VERSION	6
3.2. INITIALIZATION	7
3.2.1. NOTIFICATIONS THROUGH WINDOWS MESSAGES SENT TO A WINDOWS OBJECT	8
3.2.2. NOTIFICATIONS THROUGH WINDOWS MESSAGES SENT TO A THREAD	11
3.2.3. NOTIFICATIONS THROUGH CALLBACK FUNCTIONS	13
3.3. GET THE USER PARAMETERS	16
3.4. TERMINATE.....	17
4. BEACON MANAGEMENT	18
4.1. CHANGE THE OPERATING MODE OF THE BEACON	18
4.2. READ THE STATE OF THE BEACON.....	20
4.3. RESET THE BEACON.....	22
4.4. READ THE VERSION AND CONFIGURATION OF THE BEACON.....	23
4.5. SET THE CONFIGURATION OF THE BEACON	25
4.6. READ THE BEACON ID OF THE BEACON	27
5. TRANSACTION MANAGEMENT.....	29
5.1. START THE EMISSION OF THE BST	29
5.2. STOP THE EMISION OF THE BST	31
5.3. READ THE VST DATAGRAM	32
5.4. SEND/RECEIVE DATAGRAM TO/FROM AN OBE	33

1. INTRODUCTION

This document describes the functionality and programming mechanisms provided by BeaconManager to manage the communication with a TGB v2, a TGR-SB, or a TGR-IP over RS232 or TCP/IP. For this, it provides an API (Application Programming Interface) which makes the communication between the host and the OBE fully transparent.

Any person wanting to use this library must be a programmer with a basic understanding of the C programming language.

This DLL supports the following configurations :

	TGB v2	TGR-SB	TGR-IP
RS-232	Yes	Yes, through an ATL module	No
TCP/IP	No	Yes, through an ATL-IP module	Yes

This DLL is supported and has been tested on the following environments :

- Windows 2000
- Windows XP

2. API REFERENCE

The API is divided into 3 categories :

DLL management :

- ***BCM_GetLibVersion*** : Give the version of the DLL.
- ***BCM_InitManagerXXX*** : Initialize the communication manager.
- ***BCM_GetUserParams*** : Retrieve the user parameters passed when initialising the manager.
- ***BCM_CloseManager*** : Terminate the communication and release the resources allocated.

Beacon management :

- ***BCM_ChangeMode*** : Change the operating mode of the beacon.
- ***BCM_CheckState*** : Get the state of the beacon.
- ***BCM_Reset*** : Reset the beacon.
- ***BCM_GetConfig*** : Get the version and the configuration of the beacon.
- ***BCM_SetConfig*** : Set the configuration of the beacon.
- ***BCM_GetBeaconID*** : Get the Beacon ID of the beacon.

Transaction management :

- ***BCM_StartBST*** : Start the periodical emission of the BST.
- ***BCM_StopBST*** : Stop the periodical emission of the BST.
- ***BCM_GetVST*** : Get the VST datagram.
- ***BCM_SendCmd*** : Send a request to the OBE and wait for the response.

All functions, except ***BCM_StartBST()*** are synchronous, that means that when the function is called, the function returns after the operation is finished, and an error code gives the result of the operation. There is a timeout of 1 second for the beacon to respond to a request.

The function ***BCM_StartBST()*** is asynchronous, that means that when the function is called, the function returns before the operation is finished. In this case, the operation is performed by the library asynchronously, and the library will send a notification (through callback functions) when the operation is finished.

3. DLL MANAGEMENT

3.1. Read the DLL version

This function is used to get the version of the DLL.

Function name : **BCM_GetLibVersion()**

Function prototype : `DWORD BCM_GetLibVersion( void )`

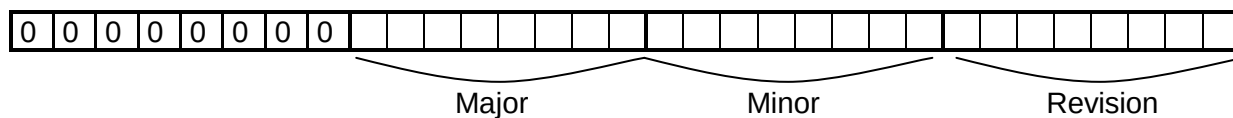
Parameters :

Name	Description
<i>Void</i>	No parameter.

Return values :

Name	Description
<i>DWORD</i>	Version of the DLL.

Format :



3.2. Initialization

Depending on the programming language you are using, and the way you want to receive asynchronous data from the DLL, you must use one of the following initialization functions :

- ***BCM_InitManagerWND*** : To receive notifications through windows messages sent to a windows object.
- ***BCM_InitManagerTHD*** : To receive notifications through windows messages sent to a thread.
- ***BCM_InitManagerFNC*** : To receive notifications via callback functions

To use a TGR-SB through an ATL-IP rather through an ATL, or to use a TGR-IP, you must use one of the following initialisation functions instead of the above initialization functions :

- ***BCM_InitManagerWND_IP*** : Same as *BCM_InitManagerWND* except that the serial port parameters are replaced by TCP/IP parameters.
- ***BCM_InitManagerTHD_IP*** : Same as *BCM_InitManagerTHD* except that the serial port parameters are replaced by TCP/IP parameters.
- ***BCM_InitManagerFNC_IP*** : Same as *BCM_InitManagerFNC* except that the serial port parameters are replaced by TCP/IP parameters.

Thoses functions initializes the DLL, and starts the communication with the beacon. You must call one of those functions once and only once, prior to any other functions.

As soon as the manager is initialized, a polling of the beacon state is performed periodically, and notifications can be sent to the application.

Note : When using the TGR-SB through an ATL-IP, or the TGR-IP, the application must wait for the connection to be established before sending any command (otherwise the command will be refused, and will return SOCK_NotConnected). The connection state is given through the notification mechanism.

Note : This DLL can be used to communicate with several beacons (one per communication port).

3.2.1. Notifications through windows messages sent to a windows object

Function name : **BCM_InitManagerWND()**
BCM_InitManagerWND_IP()

Function prototype :

```

BCM_ERR BCM_InitManagerWND(
    ST_BCM_REG_PTR *argBcmRegPtr,
    DWORD argRegNumber,
    VOID *argUserParamPtr,
    BYTE argPort,
    BYTE argBaudRate,
    BCM_STATION argStation,
    DWORD argCheckBeacon,
    BOOL argSendEvtPollingOK,
    UINT argMsg,
    HWND argWindowsHdl )

BCM_ERR BCM_InitManagerWND_IP(
    ST_BCM_REG_PTR *argBcmRegPtr,
    DWORD argRegNumber,
    VOID *argUserParamPtr,
    char *argAddr,
    WORD argPort,
    BCM_STATION argStation,
    DWORD argCheckBeacon,
    BOOL argSendEvtPollingOK,
    UINT argMsg,
    HWND argWindowsHdl )

```

Parameters :

Name	Description
<i>*argBcmRegPtr</i>	Address of a manager structure pointer. This structure will be allocated and initialized by the DLL.
<i>argRegNumber</i>	User registration number. It is not used internally and is free for use by the application. Can be set to 0.
<i>*argUserParamPtr</i>	User parameter. It is not used internally and is free for use by the application. Can be set to NULL.
<i>argPort</i> (RS232)	Serial port number on which is connected the beacon.
<i>argBaudRate</i> (RS232)	Serial port speed : • <i>BCM_CFG_9600</i> : 9600 bauds • <i>BCM_CFG_19200</i> : 19200 bauds • <i>BCM_CFG_38400</i> : 38400 bauds • <i>BCM_CFG_57600</i> : 57600 bauds • <i>BCM_CFG_115200</i> : 115200 bauds
<i>*argAddr</i> (IP)	IP address of the beacon.
<i>argPort</i> (IP)	IP port of the beacon.
<i>argStation</i>	Should be set to <i>BCM_Secondary</i> .
<i>argCheckBeacon</i>	Timer period for the polling of the beacon state (in ms). If <i>argCheckBeacon</i> = 0, then the polling is disabled.

Name	Description
<i>argSendEvtPollingOK</i>	Should the state OK of the beacon be sent to the application (for this, the polling of the state must be enabled : <i>argCheckBeacon</i> > 0).
<i>argMsg</i>	Windows message number to be sent by the DLL to the application (must be >= WM_USER (1024)). Note that the DLL is using 2 messages : <i>argMsg</i> : To notify the reception of a VST or an error. <i>argMsg+1</i> : To notify events and alarms.
<i>argWindowsHdl</i>	Handle of the windows object that will receive the message.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error when creating the manager
<i>BCM_ErrCreateEvent</i>	Error when creating the internal events
<i>BCM_ErrCreateMutex</i>	Error when creating the internal mutex
<i>BCM_ErrCreateTimer</i>	Error when creating the internal timer
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error

Notification mechanism

Depending on the situation, the DLL will notify the application using two different windows messages :

- *argMsg* : This message indicates that the DLL has received a VST, or that an error has occurred. Here *wParam* contains :
 - *BCM_CB_IN* : A VST has been received, *lParam* is not used.
 - *BCM_CB_ERR* : An error has occurred, and *lParam* contains the error code.
- *argMsg+1* : This message indicates that an alarm or an event has occurred. Here *wParam* contains :
 - *BCM_AlarmPeriph* : The beacon is not connected. This alarm is always enabled.
 - *BCM_AlarmBeacon* : The beacon is out of order. To enable this alarm, the parameter *argCheckBeacon* should be > 0.
 - *BCM_EventReset* : The beacon has been reset. To enable this event, the parameter *argCheckBeacon* should be > 0.

- *BCM_EventPollingOK*: The beacon is OK. To enable this event, the parameter *argCheckBeacon* should be > 0 and the parameter *argSendEvtPollingOK* should be set to *TRUE*.

Note : The difference between an event and an alarm, is that an alarm appears and then disappears, whereas an event only appears. In other words, an event is stateless whereas an alarm has state.

- For an alarm, *lParam* is set to *TRUE* when it appears, and *lParam* is set to *FALSE* when it disappears.
- For an event, *argState* is not used.

Important : When using *BCM_InitManagerXXX_IP*, the application **must wait** for the connection to be established before sending any command (otherwise the command will be refused, and will return *SOCK_NotConnected*). The connection state is given by the alarm *BCM_AlarmPeriph*.

3.2.2. Notifications through windows messages sent to a thread

Function name : **BCM_InitManagerTHD()**
BCM_InitManagerTHD_IP()

Function prototype :

```

BCM_ERR BCM_InitManagerTHD(
    ST_BCM_REG_PTR *argBcmRegPtr,
    DWORD argRegNumber,
    VOID *argUserParamPtr,
    BYTE argPort,
    BYTE argBaudRate,
    BCM_STATION argStation,
    DWORD argCheckBeacon,
    BOOL argSendEvtPollingOK,
    UINT argMsg,
    DWORD argThreadId )

BCM_ERR BCM_InitManagerTHD_IP(
    ST_BCM_REG_PTR *argBcmRegPtr,
    DWORD argRegNumber,
    VOID *argUserParamPtr,
    char *argAddr,
    WORD argPort,
    BCM_STATION argStation,
    DWORD argCheckBeacon,
    BOOL argSendEvtPollingOK,
    UINT argMsg,
    HWND argThreadId )

```

Parameters :

Name	Description
<i>*argBcmRegPtr</i>	Address of a manager structure pointer. This structure will be allocated and initialized by the DLL.
<i>argRegNumber</i>	User registration number. It is not used internally and is free for use by the application. Can be set to 0.
<i>*argUserParamPtr</i>	User parameter. It is not used internally and is free for use by the application. Can be set to NULL.
<i>argPort</i> (RS232)	Serial port number on which is connected the beacon.
<i>argBaudRate</i> (RS232)	Serial port speed : • <i>BCM_CFG_9600</i> : 9600 bauds • <i>BCM_CFG_19200</i> : 19200 bauds • <i>BCM_CFG_38400</i> : 38400 bauds • <i>BCM_CFG_57600</i> : 57600 bauds • <i>BCM_CFG_115200</i> : 115200 bauds
<i>*argAddr</i> (IP)	IP address of the beacon.
<i>argPort</i> (IP)	IP port of the beacon.
<i>argStation</i>	Should be set to <i>BCM_Secondary</i> .
<i>argCheckBeacon</i>	Timer period for the polling of the beacon state (in ms). If <i>argCheckBeacon</i> = 0, then the polling is disabled.

Name	Description
<i>argSendEvtPollingOK</i>	Should the state OK of the beacon be sent to the application (for this, the polling of the state must be enabled : <i>argCheckBeacon</i> > 0).
<i>argMsg</i>	Windows message number to be sent by the DLL to the application (must be >= WM_USER (1024)). Note that the DLL is using 2 messages : • <i>argMsg</i> : To notify the reception of a VST or an error. • <i>argMsg+1</i> : To notify events and alarms.
<i>argThreadId</i>	Id of the thread that will receive the message.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error when creating the manager
<i>BCM_ErrCreateEvent</i>	Error when creating the internal events
<i>BCM_ErrCreateMutex</i>	Error when creating the internal mutex
<i>BCM_ErrCreateTimer</i>	Error when creating the internal timer
<i>COMM_XXX</i>	Serial port error
<i>SOCK_XXX</i>	Socket error

Notification mechanism

See § 3.2.1 *Notifications through windows messages sent to a windows object*

3.2.3. Notifications through callback functions

Function name : **BCM_InitManagerFNC()**
BCM_InitManagerFNC_IP()

Function prototype :

```

BCM_ERR BCM_InitManagerFNC(
    ST_BCM_REG_PTR *argBcmRegPtr,
    DWORD argRegNumber,
    VOID *argUserParamPtr,
    BYTE argPort,
    BYTE argBaudRate,
    BCM_STATION argStation,
    DWORD argCheckBeacon,
    BOOL argSendEvtPollingOK,
    BCM_CB_HANDLER argCallBackProc,
    BCM_ALARM_HANDLER argAlarmProc)

BCM_ERR BCM_InitManagerFNC_IP(
    ST_BCM_REG_PTR *argBcmRegPtr,
    DWORD argRegNumber,
    VOID *argUserParamPtr,
    char *argAddr,
    WORD argPort,
    BCM_STATION argStation,
    DWORD argCheckBeacon,
    BOOL argSendEvtPollingOK,
    BCM_CB_HANDLER argCallBackProc,
    BCM_ALARM_HANDLER argAlarmProc )

```

Parameters :

Name	Description
<i>*argBcmRegPtr</i>	Address of a manager structure pointer. This structure will be allocated and initialized by the DLL.
<i>argRegNumber</i>	User registration number. It is not used internally and is free for use by the application. Can be set to 0.
<i>*argUserParamPtr</i>	User parameter. It is not used internally and is free for use by the application. Can be set to NULL.
<i>argPort</i> (RS232)	Serial port number on which is connected the beacon.
<i>argBaudRate</i> (RS232)	Serial port speed : • <i>BCM_CFG_9600</i> : 9600 bauds • <i>BCM_CFG_19200</i> : 19200 bauds • <i>BCM_CFG_38400</i> : 38400 bauds • <i>BCM_CFG_57600</i> : 57600 bauds • <i>BCM_CFG_115200</i> : 115200 bauds
<i>*argAddr</i> (IP)	IP address of the beacon.
<i>argPort</i> (IP)	IP port of the beacon.
<i>argStation</i>	Should be set to <i>BCM_Secondary</i> .
<i>argCheckBeacon</i>	Timer period for the polling of the beacon state (in ms). If <i>argCheckBeacon</i> = 0, then the polling is disabled.

Name	Description
<i>argSendEvtPollingOK</i>	Should the state OK of the beacon be sent to the application (for this, the polling of the state must be enabled : <i>argCheckBeacon</i> > 0).
<i>argCallbackProc</i>	Callback function to notify the reception of a VST or an error.
<i>argAlarmProc</i>	Callback function to notify events and alarms.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error when creating the manager
<i>BCM_ErrCreateEvent</i>	Error when creating the internal events
<i>BCM_ErrCreateMutex</i>	Error when creating the internal mutex
<i>BCM_ErrCreateTimer</i>	Error when creating the internal timer
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error

Notification mechanism

Depending on the situation, the DLL will notify the application using two different callback functions :

```
void (*BCM_CB_HANDLER) (      ST BCM REG_PTR   argBcmRegPtr,
                             BCM_CALLBACK        argTypeCB,
                             DWORD               argParam );

void (*BCM_ALARM_HANDLER) (   ST BCM REG_PTR   argBcmRegPtr,
                              BCM_ALARMS       argAlarm,
                              DWORD             argState );
```

- ***BCM_CB_HANDLER*** : This callback is used to indicate that the DLL has received a VST, or that an error has occurred. Here *argTypeCB* contains :
 - *BCM_CB_IN* : A VST has been received, *argParam* is not used.
 - *BCM_CB_ERR* : An error has occurred, and *argParam* contains the error code.
- ***BCM_ALARM_HANDLER*** : This callback indicates that an alarm or an event has occurred. Here *argAlarm* contains :
 - *BCM_AlarmPeriph* : The beacon is not connected. This alarm is always enabled.
 - *BCM_AlarmBeacon* : The beacon is out of order. To enable this alarm, the parameter *argCheckBeacon* should be > 0.

- *BCM_EventReset* : The beacon has been reset. To enable this event, the parameter *argCheckBeacon* should be > 0 .
- *BCM_EventPollingOK* : The beacon is OK. To enable this event, the parameter *argCheckBeacon* should be > 0 and the parameter *argSendEvtPollingOK* should be set to *TRUE*.

Note : The difference between an event and an alarm, is that an alarm appears and then disappears, whereas an event only appears. In other words, an event is stateless whereas an alarm has state.

- For an alarm, *argState* is set to *TRUE* when it appears, and *argState* is set to *FALSE* when it disappears.
- For an event, *argState* is not used.

Important : When using *BCM_InitManagerXXX_IP*, the application **must wait** for the connection to be established before sending any command (otherwise the command will be refused, and will return *SOCK_NotConnected*). The connection state is given by the alarm *BCM_AlarmPeriph*.

Important : As those callback functions are directly called by the internal thread which is managing the communication with the beacon, the application should return as quickly as possible.

The application should also **never** call any of the DLL functions, except *BCM_GetVST()* and *BCM_GetUserParams()*, from within a callback function, because this will generate a dead lock.

3.3. Get the user parameters

This function is used to retrieve the user parameters passed when initializing the manager. This can be used from within a callback function.

Function name : *BCM_GetUserParams()*

Function prototype : *BCM_ERR BCM_GetUserParams (*
ST_BCM_REG_PTR argBcmRegPtr,
*DWORD *argRegNumberPtr,*
*VOID **argUserParamPtr)*

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.
<i>*argRegNumberPtr</i>	Address of the user registration number. This will be filled by the library.
<i>**argUserParamPtr</i>	Address of the user parameter. This will be filled by the library.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter

3.4. Terminate

This function is used to close the communication with the beacon, and release all resources allocated.

This function should be called once at the end of the application, and no other function should be called after this.

Function name : ***BCM_CloseManager()***

Function prototype : *BCM_ERR BCM_CloseManager(ST_BCM_REG_PTR *argBcmRegPtr)*

Parameters :

Name	Description
<i>*argBcmRegPtr</i>	Address of the manager structure pointer. The structure will be destroyed and the pointer will be set to NULL by the DLL.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error

4. BEACON MANAGEMENT

4.1. Change the operating mode of the beacon

This function is used to change the operating mode of the beacon.

By default, the beacon is set in Stopped mode (*BCM_MOD_Stopped*) after power on, or after a call to **BCM_Reset**. In this mode it is not possible to communicate with any OBE. This mode is used to read or change the configuration parameters of the beacon.

To start a transaction with an OBE, the beacon should first be set in Transparent mode (*BCM_MOD_Transparent*). In this mode, all datagrams exchanged are directly transported from the host to the OBE and vice versa. There is an exception to this is when using the function **BCM_StartBST** (see §5.1 *Start the emission of the BST* for mode details).

There is a third operating mode, called Maintenance mode (*BCM_MOD_Maintenance*). This mode is reserved, and in this mode, only the function **BCM_Reset** can be used to reset the beacon.

Note 1 : It is only possible to change the operating mode, when there is no transaction in progress.

Note 2 : A transaction starts as soon as the beacon has received a VST from an OBE, and ends when the beacon has received a request from the host with the last command flag set.

Function name : **BCM_ChangeMode** ()

Function prototype : *BCM_ERR BCM_ChangeMode (*
ST_BCM_REG_PTR argBcmRegPtr,
BYTE argMode)

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.
<i>ArgMode</i>	Operating mode : <i>BCM_MOD_Stopped</i> : To switch off the HF, and to read or change the parameters <i>BCM_MOD_Transparent</i> : To allow a transaction <i>BCM_MOD_Maintenance</i> : Should not be used

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error while processing the request
<i>BCM_CommAborted</i>	The communication have been aborted
<i>BCM_CommTimeout</i>	The beacon did not respond to the request
<i>BCM_ErrResponse</i>	The response received from the beacon is not correct
<i>BCM_ErrEvent</i>	Internal event error while processing the request
<i>BCM_CdeRefused</i>	Command refused because the operating mode is not correct, or because the datagram sent is not correct
<i>BCM_TrxInProgress</i>	Command refused because there is a transaction in progress
<i>BCM_PbBeacon</i>	Command refused because the beacon is out of order and the mode requested is not <i>BCM_MOD_Stopped</i>
<i>BCM_NotConfig</i>	Command refused because the beacon has not been configured
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error

4.2. Read the state of the beacon

This function is used to get the internal state of the beacon.

If this function succeeds, the following state structure is filled :

```
struct ST_BCM_STATE
{
    BYTE state;
    BYTE mode;
    BYTE trxInProgress;
};
```

- *state* : gives the state of the beacon :
 - *BCM_NoErr* : OK,
 - *BCM_PbBeacon* : The beacon is out of order,
 - *BCM_ResetBeacon* : The beacon has been reset.
- *mode* : gives the operating mode of the beacon (see §4.1 *Change the operating mode of the beacon*) :
 - *BCM_MOD_Stopped*
 - *BCM_MOD_Transparent*
 - *BCM_MOD_Maintenance*
- *trxInProgress* : This flag is set to 1 when a transaction is in progress, otherwise it is set to 0.

Note 1 : This function can be used in any operating mode.

Note 2 : This function can be automatically and periodically requested by the library (see §3.2 *Initialization*).

Note 3 : This state is internally refreshed by the TGR-SB every 100 ms.

Note 4 : Be aware that the state *BCM_ResetBeacon* is only returned once for each reset. And it is always returned through the notification mechanism.

Function name : ***BCM_CheckState()***

Function prototype : *BCM_ERR BCM_CheckState(*
ST_BCM_REG_PTR argBcmRegPtr,
ST_BCM_STATE_PTR argStatePtr)

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.
<i>ArgStatePtr</i>	Pointer on the state structure that will be filled by the function.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error while processing the request
<i>BCM_CommAborted</i>	The communication have been aborted
<i>BCM_CommTimeout</i>	The beacon did not respond to the request
<i>BCM_ErrResponse</i>	The response received from the beacon is not correct
<i>BCM_ErrEvent</i>	Internal event error while processing the request
<i>BCM_CdeRefused</i>	Command refused because the datagram sent is not correct
<i>BCM_NotConfig</i>	Command refused because the beacon has not been configured
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error

4.3. Reset the beacon

This function is used to perform a software reset of the beacon.

Note 1 : This function can be used in any operating mode.

Note 2 : After a reset, the operating mode of the beacon is reset to Stopped mode.

Note 3 : After a reset, and during the next call to **BCM_CheckState**, the beacon will set only once the field *ST_BCM_STATE.state* to *BCM_ResetBeacon*.

Function name : **BCM_Reset()**

Function prototype : *BCM_ERR BCM_Reset (*
ST_BCM_REG_PTR argBcmRegPtr)

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error while processing the request
<i>BCM_CommAborted</i>	The communication have been aborted
<i>BCM_CommTimeout</i>	The beacon did not respond to the request
<i>BCM_ErrResponse</i>	The response received from the beacon is not correct
<i>BCM_ErrEvent</i>	Internal event error while processing the request
<i>BCM_CdeRefused</i>	Command refused because the operating mode is not correct, or because the datagram sent is not correct
<i>BCM_NotConfig</i>	Command refused because the beacon has not been configured
<i>COMM_XXX</i>	Serial port error
<i>SOCK_XXX</i>	Socket error

4.4. Read the version and configuration of the beacon

This function is used to read the version and the configuration parameters of the beacon.

If this function succeeds, the following configuration structure is filled :

```
struct ST_BCM_CONFIG
{
    BYTE    version[255+1];

    // BST_VST parameters
    WORD    manufacturerID;
    DWORD   individualID;
    WORD    versionDescam;

    // Layer 2 I2C2 parameters
    WORD    nbRetriesBST;
    WORD    timeoutRetryBST;
    WORD    nbRetriesACn;
    BYTE    frequency;

    // Layer 2 BAC parameters
    WORD    baudRate;
    WORD    watchdog;
    WORD    technology;

    // MARTEL parameters
    BYTE    nbBeacons;
    WORD    numLocation;

    BYTE    dummy[4];
};
```

To get a complete description of those fields, please refer to the PERTEL documentation.
For the TGB v2, TGR-SB and TGR-IP, only some fields are used, all others are set to 0 :

- *version*: A string terminated by '\0' that gives the name and the version of the beacon.
- *manufacturerID*: The manufacturer ID of the beacon.
- *individualID*: The individual ID of the beacon.
- *timeoutRetryBST*: Emission period of the BST (expressed in µs).
- *nbRetriesACn*: Number of retries after timeout Can.
- *frequency*: Frequency of the beacon :
 - *BCM_CFG_F1*
 - *BCM_CFG_F2*
 - *BCM_CFG_F3*
 - *BCM_CFG_F4*

- *baudRate* :
 - *BCM_CFG_9600*
 - *BCM_CFG_19200*
 - *BCM_CFG_38400*
 - *BCM_CFG_57600*
 - *BCM_CFG_115200*

Note 1 : This function can only be used in Stopped or Maintenance mode.

Function name : *BCM_GetConfig()*

Function prototype : *BCM_ERR BCM_GetConfig(ST_BCM_REG_PTR argBcmRegPtr, ST_BCM_CONFIG_PTR argConfigPtr)*

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.
<i>ArgConfigPtr</i>	Pointer on the configuration structure that will be filled by the function.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error while processing the request
<i>BCM_CommAborted</i>	The communication have been aborted
<i>BCM_CommTimeout</i>	The beacon did not respond to the request
<i>BCM_ErrResponse</i>	The response received from the beacon is not correct
<i>BCM_ErrEvent</i>	Internal event error while processing the request
<i>BCM_CdeRefused</i>	Command refused because the operating mode is not correct, or because the datagram sent is not correct
<i>BCM_NotConfig</i>	Command refused because the beacon has not been configured
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error

4.5. Set the configuration of the beacon

This function is used to set the frequency and the serial port speed of the beacon.

Note 1 : This function can only be used in Stopped or Maintenance mode.

Note 2 : The modification of the serial port speed of the beacon **does not** change the serial port speed of the host. So it is up to the programmer to close and reinitialise the BeaconManager with the correct serial port speed.

Warning : Even when using this function with a TGR-SB beacon connected to a host through an ATL-IP, the serial port speed must be specified. This is because the speed is stored in the beacon, and will be used, if later the ATL-IP is replaced by an ATL (RS232 communication).
For TGR-IP, the serial port speed must also be specified to a correct value (*BCM_CFG_115200*), even if it is not used.

Function name : **BCM_SetConfig()**

Function prototype : *BCM_ERR BCM_SetConfig(*
ST_BCM_REG_PTR argBcmRegPtr,
BYTE argMode,
BYTE argFrequency,
BYTE argBaudRate)

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.
<i>argMode</i>	Taken into account mode : <i>BCM_CFG_Immediate</i> : This mode indicates that the configuration parameter should be taken into account strait away. This will generate a reset of the beacon as soon as the function returns. <i>BCM_CFG_Reset</i> : This mode indicates that the configuration parameter should be taken into account after the next reset.
<i>argFrequency</i>	Frequency requested : <i>BCM_CFG_F1</i> <i>BCM_CFG_F2</i> <i>BCM_CFG_F3</i> <i>BCM_CFG_F4</i>
<i>argBaudRate</i>	Serial port baud rate requested : <i>BCM_CFG_9600</i> : 9600 bauds <i>BCM_CFG_19200</i> : 19200 bauds <i>BCM_CFG_38400</i> : 38400 bauds <i>BCM_CFG_57600</i> : 57600 bauds <i>BCM_CFG_115200</i> : 115200 bauds

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error while processing the request
<i>BCM_CommAborted</i>	The communication have been aborted
<i>BCM_CommTimeout</i>	The beacon did not respond to the request
<i>BCM_ErrResponse</i>	The response received from the beacon is not correct
<i>BCM_ErrEvent</i>	Internal event error while processing the request
<i>BCM_CdeRefused</i>	Command refused because the operating mode is not correct, or because the datagram sent is not correct
<i>BCM_PbBeacon</i>	Command refused because the beacon is out of order
<i>BCM_PbParam</i>	Command refused because a parameter is not correct
<i>BCM_NotConfig</i>	Command refused because the beacon has not been configured
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error

4.6. Read the Beacon ID of the beacon

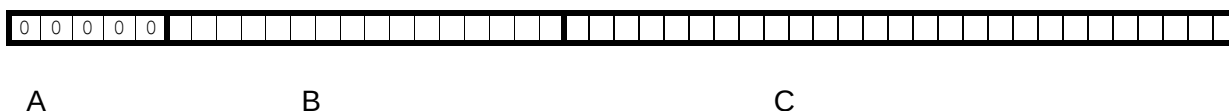
This function is used to get the Beacon ID sent in the last BST. This could be useful in personalization process, to calculate security algorithms of certain kind of OBE.

The Beacon ID returned depends on the operating mode of the beacon and the type of BST :

- In Stopped or Maintenance mode, the Beacon ID returned corresponds to the Beacon ID that is configured in the beacon.
- In Transparent mode, the Beacon ID returned depends on the type of BST (i.e. the parameter *argTypeBST* of the function **BCM_StartBST**) :
 - With *BCM_BST_Normal*, then the Beacon ID corresponds also to the Beacon ID that is configured in the beacon.
 - With *BCM_BST_ChangeBID*, then the BeaconID corresponds to the last Beacon ID sent (the lowest 3 bits of the Individual ID are automatically incremented modulo 8 every 128 emissions (see §5.1 *Start the emission of the BST* for more details)).

Note 1 : The size of a Beacon ID is *BCM_SIZE_BEACONID* bytes.

The Beacon ID is composed like this :



- A : padding (5 bits).
- B : Manufacturer ID (16 bits).
- C : Individual ID (27 bits).

Function name : **BCM_GetBeaconID()**

Function prototype : *BCM_ERR BCM_GetBeaconID(ST_BCM_REG_PTR argBcmRegPtr, BYTE *argBeaconIDPtr)*

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.
<i>*argBeaconIDPtr</i>	Pointer on a buffer that will be filled with the Beacon ID.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error while processing the request
<i>BCM_CommAborted</i>	The communication have been aborted
<i>BCM_CommTimeout</i>	The beacon did not respond to the request
<i>BCM_ErrResponse</i>	The response received from the beacon is not correct
<i>BCM_ErrEvent</i>	Internal event error while processing the request
<i>BCM_CdeRefused</i>	Command refused because the datagram sent is not correct
<i>BCM_NotConfig</i>	Command refused because the beacon has not been configured
<i>COMM_XXX</i>	Serial port error
<i>SOCK_XXX</i>	Socket error

5. TRANSACTION MANAGEMENT

5.1. Start the emission of the BST

This function is used to start the periodical emission of the BST.

As this function is asynchronous, on return, it gives an error code that indicates if the command has been successfully sent to the beacon.

As soon as the beacon receives a VST, a transaction is started, and the library notifies the application through the notification mechanism. The application can then call the function ***BCM_GetVST*** to get the VST datagram.

Depending on the application, it might be interesting to bypass the anti-passback mechanism of the OBE. To do this, the solution is to modify the Beacon ID of the beacon by setting the parameter *argTypeBST* to *BCM_BST_ChangeBID*. In this case, every 128 emissions of the BST, the beacon will automatically increment modulo 8 the lowest 3 bits of the Individual ID of the beacon.

Note 1 : It is only possible to use this function in Transparent mode, and only if there is no transaction in progress.

Note 2 : A transaction starts as soon as the beacon has received a VST from an OBE, and ends when the beacon has received a request from the host with the last command flag set.

Note 3 : In Transparent mode, the beacon is fully transparent, this means that all datagrams are transported without any modification from the host to the OBE and vice versa. But there is one exception to this : The Beacon ID field of the BST. In fact, the beacon ID is completely managed by the beacon, and so, each time a host sends a BST datagram, the beacon overwrites the Beacon ID field.

Function name : *BCM StartBST()*

```

Function prototype : BCM_ERR      BCM_StartBST(
                                     ST_BCM_REG_PTR      argBcmRegPtr,
                                     BYTE                  *argDatagram,
                                     DWORD                  argLgDatagram,
                                     BYTE                  argTypeBST )

```

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.
<i>*argDatagram</i>	Pointer on the BST datagram (see ISO 14906)
<i>argLgDatagram</i>	Size of the BST datagram (it can never be longer than <i>BCM_SIZEMAX_BST</i> bytes).
<i>ArgTypeBST</i>	Type of BST :

Name	Description
	<i>BCM_BST_Normal</i> : The Individual ID is never modified. <i>BCM_BST_ChangeBID</i> : The lowest 3 bits of the Individual ID are incremented every 128 emissions.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error while processing the request
<i>BCM_CommAborted</i>	The communication have been aborted
<i>BCM_CommTimeout</i>	The beacon did not respond to the request
<i>BCM_ErrResponse</i>	The response received from the beacon is not correct
<i>BCM_ErrEvent</i>	Internal event error while processing the request
<i>BCM_CdeRefused</i>	Command refused because the operating mode is not correct, or because the datagram sent is not correct
<i>BCM_TrxInProgress</i>	Command refused because there is a transaction in progress
<i>BCM_PbBeacon</i>	Command refused because the beacon is out of order
<i>BCM_NotConfig</i>	Command refused because the beacon has not been configured
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error

5.2. Stop the emission of the BST

This function is used to stop the periodical emission of the BST.

Note : This function can only be used in Transparent mode, and only if there is no transaction in progress.

Function name : **BCM_StopBST()**

Function prototype : *BCM_ERR BCM_StopBST(*
ST_BCM_REG_PTR argBcmRegPtr)

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error while processing the request
<i>BCM_CommAborted</i>	The communication have been aborted
<i>BCM_CommTimeout</i>	The beacon did not respond to the request
<i>BCM_ErrResponse</i>	The response received from the beacon is not correct
<i>BCM_ErrEvent</i>	Internal event error while processing the request
<i>BCM_CdeRefused</i>	Command refused because the operating mode is not correct, or because the datagram sent is not correct
<i>BCM_TrxInProgress</i>	Command refused because there is a transaction in progress
<i>BCM_NotConfig</i>	Command refused because the beacon has not been configured
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error

5.3. Read the VST datagram

This function is used to get the VST datagram.

Note : This function should only be used after the application has been notified by the DLL that it has received the VST of an OBE.

Function name : **BCM_GetVST()**

Function prototype : *BCM_ERR BCM_GetVST(*
ST_BCM_REG_PTR argBcmRegPtr,
*BYTE *argDatagram,*
*DWORD *argLgDatagram,*
DWORD argLgMaxDatagram)

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.
<i>*argDatagram</i>	Pointer on the VST datagram buffer. This will be filled by the DLL.
<i>*argLgDatagram</i>	Pointer on the VST datagram size. This will also be filled by the DLL.
<i>ArgLgMaxDatagram</i>	Size of the VST datagram buffer. Note : a VST datagram can never be longer than <i>BCM_SIZEMAX_ANSWER</i> , so the minimum size for the VST datagram buffer should be <i>BCM_SIZEMAX_ANSWER</i> bytes.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter

5.4. Send/receive datagram to/from an OBE

This function is used to send a request to an OBE, and wait for the response.

The parameter *argIsLast* indicates if it is the last request, and is used by the beacon to close the transaction.

Note : It is necessary to send a request at the end of a transaction to close it, even if an error occurs.

Warning : If the function returns *BCM_TmoOBE*, that means that the beacon has lost the OBE, but there is no way to tell if the request has been successfully processed or not by the OBE. So an application should be aware of this, in particular when using a request that modify an attribute after each execution. Those requests are the following :

- DEBIT, CREDIT, ADD, SUBSTRACT,
- SET/SET_STAMPED on multi-instance attributes.

Note 1 : This function can not be used to send a BST.

Note 2 : This function can only be used when there is a transaction in progress.

Function name : *BCM_SendCmd()*

Function prototype : *BCM_ERR BCM_SendCmd(ST_BCM_REG_PTR BYTE DWORD BYTE DWORD DWORD BOOL argBcmRegPtr, *argDatagramOut, argLgDatagramOut, *argDatagramIn, *argLgDatagramIn, argLgMaxDatagramIn, argIsLast)*

Parameters :

Name	Description
<i>argBcmRegPtr</i>	Pointer on the manager structure.
<i>ArgDatagramOut</i>	Pointer on the request datagram buffer (see ISO 14906).
<i>ArgLgDatagramOut</i>	Size of the request datagram
<i>*argDatagramIn</i>	Pointer on the response datagram buffer. This will be filled by the library.
<i>*argLgDatagramIn</i>	Pointer on the response datagram size. This will also be filled by the library.
<i>ArgLgMaxDatagramIn</i>	Size of the response datagram buffer. Note : a response datagram can never be longer than <i>BCM_SIZEMAX_ANSWER</i> , so the minimum size for the response buffer should be <i>BCM_SIZEMAX_ANSWER</i> bytes.
<i>ArgIsLast</i>	Indicates if it is the last request.

Return values :

Name	Description
<i>BCM_NoErr</i>	Function executed with success
<i>BCM_BadParam</i>	Bad parameter
<i>BCM_ErrMemory</i>	Memory error while processing the request
<i>BCM_CommAborted</i>	The communication have been aborted
<i>BCM_CommTimeout</i>	The beacon did not respond to the request
<i>BCM_ErrResponse</i>	The response received from the beacon is not correct
<i>BCM_ErrEvent</i>	Internal event error while processing the request
<i>BCM_CdeRefused</i>	Command refused because the operating mode is not correct, or because there is no transaction in progress, or because the datagram sent is not correct
<i>BCM_PbBeacon</i>	Command refused because the beacon is out of order
<i>BCM_TmoOBE</i>	The beacon has lost the OBE.
<i>BCM_NotConfig</i>	Command refused because the beacon has not been configured
<i>COMM_xxx</i>	Serial port error
<i>SOCK_xxx</i>	Socket error